

What Makes One Memory Different From Another



Location

Inside the processor (registers, cache, main memory) or outside it (flash, disk, tape).



Capacity

How many words or bytes a memory unit can hold.



Access Method

Random (RAM), sequential (tape), direct (disk), or associative (cache), depending on the technology.

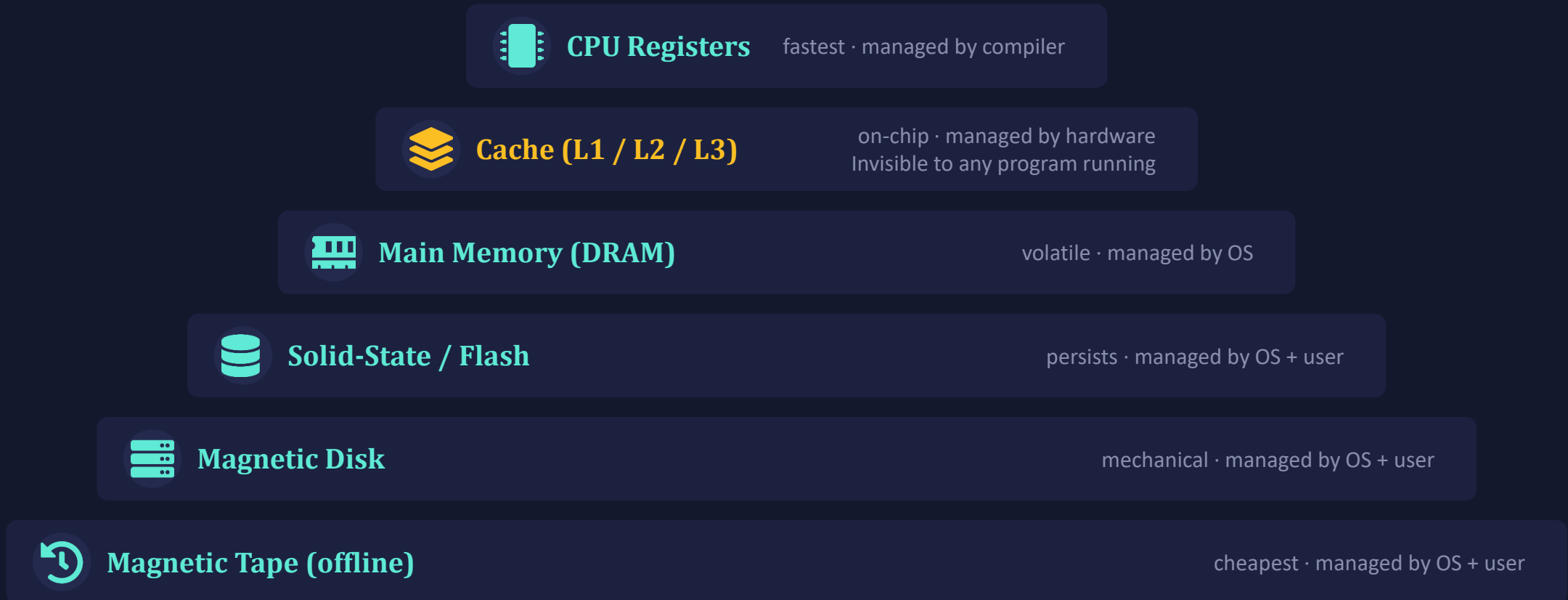


Performance

Access time, cycle time, transfer rate — this is where cache earns its keep.

Fast and Small at the Top, Slow and Huge at the Bottom

Every step down trades speed and cost per bit for raw capacity, and hands control to someone new.



Three Ideas Guide Every Memory Hierarchy



Locality

Whatever was used recently is likely to be used again soon, and whatever sits near it is likely to be needed next. This is the one assumption the entire hierarchy is a bet on.



Inclusion

Data copied into a smaller, faster level is still also present in the larger, slower level below it. Nothing lives only in the fast copy.



Coherence

If one copy of a data item changes, every other copy of it, at every other level, must eventually agree.

Today is mostly about the first one, but the names of the other two are worth knowing on sight.

Programs Don't Touch Memory Randomly



Temporal Locality

If a word was referenced recently, it is likely to be referenced again soon.

Constants, loop counters, and the working stack all lean on this.



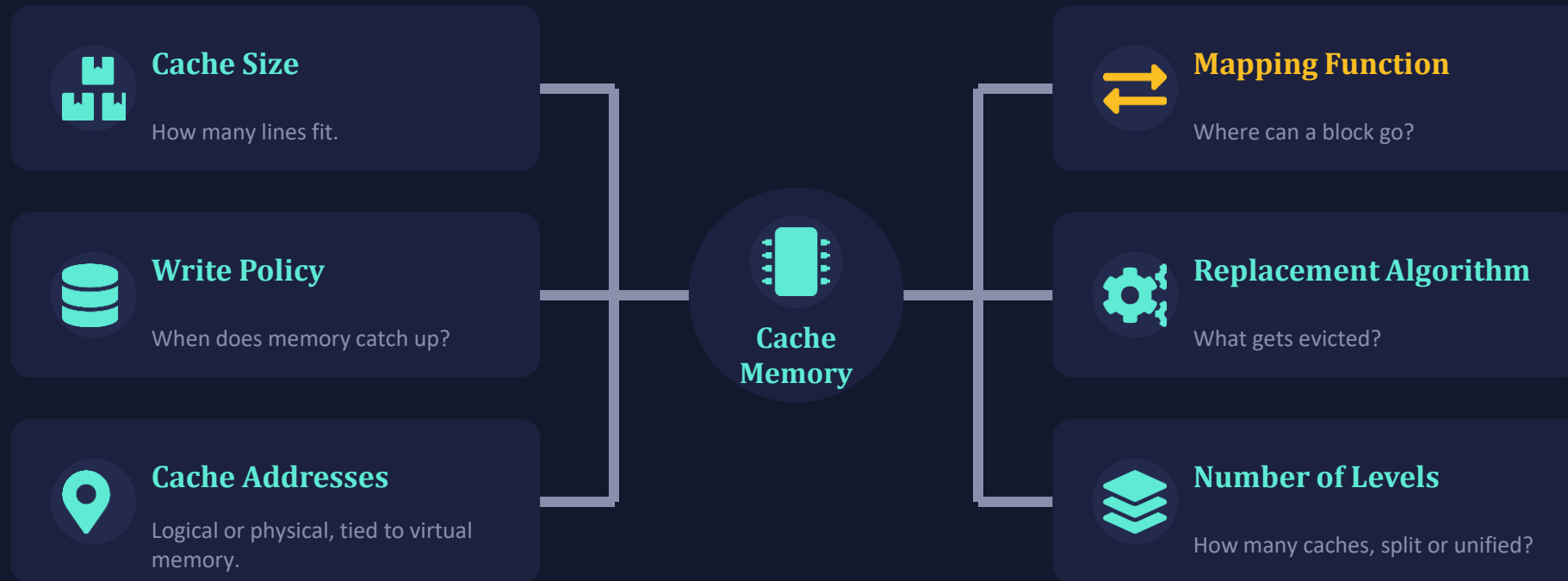
Spatial Locality

If address X was just referenced, addresses near X are likely to be referenced soon too.

Walking an array in order, or fetching the next instruction in sequence, both lean on this.

Today's live demo tests exactly this: does the order you touch memory change how fast the program runs?

Every Cache Is Built From the Same Design Decisions



How Big Is a Word, Really?

Each unit is the previous one, multiplied up: bit, byte, word, cache line, page.



Bit

1 binary digit



Byte

8 bits, one character



Word

the CPU's natural chunk, 4 or 8 bytes



Cache Line

typically 64 bytes, one fetch from memory



Page

typically 4 KB, the OS's unit of transfer

Three Ways to Place a Block in Cache



Direct Mapping

$$i = j \bmod m$$

Each block maps to exactly one cache line. Cheap and fast to check, but two active blocks that map to the same line will fight for it (thrashing).

Thrashing – imagine a program’s data straddles two different blocks and those blocks end up mapping to the same cache line as the program accesses the data.



Associative Mapping

any block, any line

A block can go in any line at all. Flexible and thrash-resistant, but the hardware must compare every line’s tag at once.



Set-Associative

any line, in one set

Split the cache into sets; a block can go in any line within its one set. The compromise nearly every real processor uses today.

How an Address Finds Its Way

MAIN MEMORY

The address is a guarantee: it names one unique, permanent location.

CACHE

The address is a hypothesis: the index names a candidate, the tag then confirms it.

The cache turns that hypothesis into a check using three fields split out of the address:



TAG

Compared against the block already in that line, to confirm it's the right one.



SET

Picks which set (or line, for direct mapping) to look in.



OFFSET

Picks the exact byte within the block.

In Part 1's example (64-byte line, 64 sets): 6 bits pick the offset, 6 bits pick the set, the rest of the address is the tag.

(segment widths above are illustrative, not to exact bit scale)

Writing to Cache, and Why L2 and L3 Exist



Write Policy

Write-through: every write updates cache and main memory. Simple, but generates more bus traffic.

Write-back: writes only touch the cache; a dirty bit marks what still needs to flush. Less traffic, more complexity.



Multilevel Caches

L1 sits on the core: private, smallest, fastest.

L2 is usually on-core too in current chips, though older designs shared it across a small cluster.

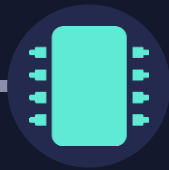
L3 is on-chip but shared by all cores: largest, slowest of the three, still far faster than main memory.

Why There's an L2 and L3 At All



External Cache Added

Main memory was slower than the system bus, so a faster external chip was added.



Cache Moves On-Chip

Rising processor speed made even the external bus a bottleneck, so cache moved onto the die.



A Second Level Added

L1 stayed small on purpose, since small caches are fast, so a bigger L2 was added to catch its misses.



A Third, Shared Level Added

With multiple cores on one chip, a shared L3 catches what any core's L2 misses.

From Slides to Your Terminal



Every idea on the last ten slides is something your machine can show you right now.

`lscpu` · `/sys/devices/system/cpu/cpu*/cache/` · `free -h` · a five-line C program